Análise de Vulnerabilidades de Memória Assistida por Agentes de IA: Um Estudo Experimental com PHP2Shell*

Adriano Araújo[1], **Artur Rizzi**[1], **Eduardo Aniceto**[1], **Izabel Chaves**[1], **Pedro Guimarães**[1]

[1] Departamento de Ciência da Computação – Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
30.535-610 – Belo Horizonte – Brasil

{1464597, 1568608, 1593221, iopchaves, 1447017}@pucminas.edu.br

Abstract. *Artificial intelligence agents can help automate software vulnerability analysis, but their effectiveness and reproducibility require further evaluation. This work proposes an experimental study of OpenCode for analyzing memory vulnerabilities, using PHP2Shell as a case study. The methodology includes a controlled environment, standardized instructions, and experimental validation of the findings. The gem5 simulator may be used to investigate relevant architectural aspects, depending on software compatibility. The evaluation will consider detection accuracy, false positives and negatives, analysis time, computational cost, and reproducibility. The goal is to identify the capabilities and limitations of AI-assisted vulnerability analysis.*

Resumo. *Agentes de inteligência artificial podem auxiliar na análise de vulnerabilidades de software, mas sua eficácia e reprodutibilidade precisam ser avaliadas. Este trabalho propõe um estudo experimental do OpenCode na análise de vulnerabilidades de memória, utilizando o PHP2Shell como estudo de caso. A metodologia inclui um ambiente controlado, instruções padronizadas e validação experimental dos resultados. O simulador gem5 poderá ser utilizado para investigar aspectos arquiteturais relevantes, conforme a compatibilidade do software. A avaliação considerará a precisão na identificação de falhas, os falsos positivos e negativos, o tempo de análise, o custo computacional e a reprodutibilidade. O objetivo é compreender as capacidades e limitações da análise de vulnerabilidades assistida por IA.*

Keywords: Artificial intelligence agents; Software security; Memory vulnerabilities; gem5; OpenCode; PHP2Shell.

1. Introdução

1.1. Contexto e Motivação

Vulnerabilidades de memória, como estouros de buffer, uso após a liberação e acessos fora dos limites, continuam entre as falhas mais críticas de software, pois podem levar à execução remota de código e ao comprometimento total do sistema. Sua análise é trabalhosa: exige a leitura de grandes bases de código, a formulação de hipóteses sobre o fluxo de dados e a construção de entradas capazes de reproduzir a falha em um ambiente controlado.

*O código-fonte deste projeto está disponível em: `https://github.com/`

Nesse contexto, agentes de inteligência artificial baseados em modelos de linguagem têm sido investigados como apoio à descoberta e à exploração de vulnerabilidades. Benchmarks recentes avaliam esses agentes em tarefas como a exploração de vulnerabilidades reais em aplicações web [Zhu et al. 2025], tarefas reais de segurança de software [Lee et al. 2025], a reprodução de vulnerabilidades do zero [He et al. 2026], o teste de intrusão [Gioacchini et al. 2025] e a execução de ciberataques de ponta a ponta [Shi et al. 2026]. Os resultados indicam um potencial relevante, mas também uma taxa de sucesso ainda limitada em cenários realistas; por exemplo, o melhor agente avaliado no CVE-Bench explorou até 13% das vulnerabilidades [Zhu et al. 2025]. Levantamentos recentes sobre agentes baseados em LLMs para a descoberta automatizada de vulnerabilidades [Addis et al. 2026] reforçam a necessidade de avaliar, com métodos controlados, a eficácia, a reprodutibilidade e o custo dessas abordagens.

Este trabalho investiga essa lacuna por meio de um estudo de caso: a análise de vulnerabilidades de memória no PHP2Shell, descrito na próxima subseção, usando o OpenCode como *harness* de agente e modelos de linguagem *open-weight* fornecidos gratuitamente pela NVIDIA. A escolha de um caso concreto, com a cadeia de exploração documentada, permite confrontar as conclusões do agente com evidências obtidas em testes controlados, em vez de depender apenas da plausibilidade de suas respostas. Complementarmente, o simulador gem5 poderá ser empregado, conforme a compatibilidade do software, para observar aspectos arquiteturais da execução, como ciclos, instruções e comportamento da hierarquia de memória.

1.2. O Caso de Estudo: PHP2Shell

O PHP2Shell é o nome dado pela equipe de pesquisa da Wiz a um conjunto de falhas de segurança de memória no interpretador PHP, divulgado de forma coordenada em 21 de outubro de 2026 [Tzadik et al. 2026]. As falhas foram registradas como CVE-2026-109618 e CVE-2026-109619, ambas classificadas como críticas (CVSS v4 de 9,2) e associadas à fraqueza CWE-416 (uso após a liberação) [PHP Group 2026a, PHP Group 2026b]. Segundo a Wiz, toda versão do PHP lançada na última década, desde a 5.4, é afetada por ao menos uma delas, e a exploração prescinde de uma aplicação vulnerável: até um simples *Hello World* em uma versão afetada pode ser comprometido [Tzadik et al. 2026]. O caso é relevante para este estudo também pela forma como foi descoberto: a Wiz as encontrou com apoio de modelos de IA, e a OpenAI relata que a Calif identificou uma das falhas com o GPT-5.6 Sol, no âmbito da iniciativa *Patch the Planet*, com a Wiz reportando ambas de forma independente [Tzadik et al. 2026, OpenAI 2026]. A própria OpenAI considera o episódio um exemplo de como a IA pode revelar falhas de impacto em escala de Internet [OpenAI 2026].

A causa comum é o tratamento de erros fatais, em especial os de falta de memória (*out-of-memory*, OOM), durante o processamento de uma requisição. No CVE-2026-109618, o PHP usa `longjmp()` para abandonar o contexto atual quando uma alocação falha; se isso ocorre antes de a memória da requisição estar totalmente inicializada, o encerramento da requisição lê ou desvia a execução com base em um estado inválido, e a alocação que falhou nem precisa estar relacionada à memória corrompida [PHP Group 2026a]. No CVE-2026-109619, o estado global mantido por um processo *worker* persistente (erros, sessão, objetos e extensões) continua apontando para memória da requisição já liberada, pois erros fatais interrompem a operação antes de o estado ser reiniciado; uma requisição posterior, atendida pelo mesmo *worker*, pode então ler ou liberar esse ponteiro pendente [PHP Group 2026b]. Na análise da Wiz, isso se

traduz em primitivas como o uso após a liberação no *unserializer* do progresso de *upload* de sessões, a divulgação de memória do processo por meio do cookie `PHPSESSID` e a confusão de tipos no OPcache, encadeadas para obter execução remota de código sem autenticação em WordPress, Drupal e Joomla [Tzadik et al. 2026].

O impacto é a divulgação de memória do *worker*, que pode conter senhas, *tokens* e chaves de API, e a execução remota de código com os privilégios do servidor web. Os avisos de segurança do PHP confirmam a exploração contra o Apache com *mod_php* e indicam que o PHP-FPM é suscetível, embora sem exploração completa demonstrada [PHP Group 2026a, PHP Group 2026b]. As correções foram publicadas nas versões 8.2.35, 8.3.36, 8.4.27 e 8.5.12, e a atualização é a única remediação completa; *flags* como `session.upload_progress.enabled=0` ou `fatal_error_backtraces=0` reduzem apenas caminhos específicos [PHP Group 2026b]. Por reunir falhas de memória reais, recentes e com divulgação detalhada, o PHP2Shell oferece um caso adequado para confrontar a análise de um agente com evidências verificáveis.

1.3. *Harness* e a Escolha do OpenCode

Um agente de IA para tarefas de software não é apenas um modelo de linguagem: o modelo é envolvido por um *harness*, o programa que o conecta ao ambiente de trabalho. É o *harness* que monta as instruções e o contexto enviados ao modelo, oferece ferramentas (como leitura e edição de arquivos e execução de comandos no terminal), aplica controles de permissão e repete o ciclo de raciocínio, ação e observação até a conclusão da tarefa. Por isso, dois experimentos com o mesmo modelo podem divergir se o *harness* for diferente, e ele deve ser fixado e registrado como uma variável do experimento, assim como o modelo.

Adotamos o OpenCode como *harness* por ser uma alternativa gratuita e de código aberto, distribuída sob a licença MIT [OpenCode 2026]. A escolha se apoia em quatro razões: (i) o custo nulo de licenciamento, que permite a qualquer grupo repetir o estudo; (ii) o código aberto, que possibilita inspecionar o funcionamento do agente e registrar a versão exata utilizada; (iii) a independência de modelo, já que o OpenCode se conecta a mais de 200 provedores, inclusive a modelos locais [OpenCode 2026], o que permite trocar o modelo sem alterar o *harness*; e (iv) a interface de terminal e o SDK para *scripts* e integração contínua, somados a agentes, *skills*, comandos e permissões configuráveis [OpenCode 2026], que favorecem instruções padronizadas e execuções automatizadas e repetíveis.

1.4. Modelos *Open-Weight*

Optamos por modelos *open-weight* por três motivos. Primeiro, o acesso gratuito aos modelos hospedados dispensa hardware próprio, como GPUs de grande capacidade, e torna o estudo acessível. Segundo, assumimos que os modelos que atendem integralmente à definição de código aberto ainda são menos numerosos e, em geral, de menor porte, o que tende a limitar o desempenho em tarefas de agente; essa premissa será examinada em trabalhos futuros. Terceiro, a publicação dos pesos permite fixar e registrar o modelo e, posteriormente, executá-lo localmente, o que é mais favorável à reprodutibilidade do que um modelo proprietário acessível apenas por uma API que pode ser alterada ou descontinuada sem aviso.

Os modelos de linguagem utilizados serão *open-weight* e acessados gratuitamente pelo catálogo *build.nvidia.com* [NVIDIA 2026e], cujos *endpoints* são oferecidos aos

membros do NVIDIA Developer Program para prototipagem, pesquisa, desenvolvimento e testes [NVIDIA 2026d]. Um modelo *open-weight* tem seus pesos (parâmetros) publicamente disponíveis, o que permite executá-lo e adaptá-lo, mas não implica a divulgação dos dados nem do código de treinamento. Já a definição de IA de código aberto da Open Source Initiative (OSI) exige, além dos parâmetros, informações suficientemente detalhadas sobre os dados de treinamento e o código completo de treinamento e execução, sob termos aprovados pela OSI [Open Source Initiative 2024]. Pela definição, os pesos isoladamente não bastam. Por isso, neste artigo, os modelos são chamados de *open-weight*, e não de *open-source*.

Serão utilizados três modelos do catálogo: `moonshotai/kimi-k3.1-pro`, descrito pela NVIDIA como um modelo aberto, voltado a programação agêntica, com 2,8 trilhões de parâmetros totais, 104 bilhões ativos e contexto de 1 milhão de *tokens*, distribuído sob uma licença própria do desenvolvedor [NVIDIA 2026c]; `deepseek-ai/deepseek-v4.1-flash`, um modelo *Mixture-of-Experts* (MoE) com 552 bilhões de parâmetros totais e 8 bilhões ativos na fase de preenchimento, cujo modelo base é licenciado sob MIT [NVIDIA 2026a]; e `z-ai/glm-5-3-flash`, um MoE multimodal com 320 bilhões de parâmetros totais e 18 bilhões ativos por *token*, também sob MIT [NVIDIA 2026b]. Os três oferecem janelas de contexto da ordem de 1 milhão de *tokens*, o que permite fornecer ao agente trechos extensos do código-fonte, e provêm de desenvolvedores distintos, o que permite comparar o efeito do modelo mantendo o *harness* fixo.

Esse arranjo tem limitações que serão consideradas na análise. O acesso gratuito é voltado à prototipagem, com limites de requisição que variam conforme o modelo e o número de usuários simultâneos [NVIDIA 2026d], o que pode afetar o tempo de análise e a reprodutibilidade. Além disso, na data de consulta, a página do Kimi indicava que o acesso por API ainda não estava disponível e que o *endpoint* era voltado apenas à prévisualização [NVIDIA 2026c]. Será preciso registrar o identificador do modelo e a data de cada execução. Por fim, as instruções e os trechos de código enviados ao agente trafegam por infraestrutura de terceiros. Como o código-fonte do PHP e os avisos de segurança analisados são públicos, isso é aceitável neste estudo, mas o arranjo não garante análises anônimas, o que motiva a proposta de trabalhos futuros.

1.5. Hipótese e Objetivos

Parte-se da hipótese de que um agente de IA, conduzido por instruções padronizadas em um ambiente isolado, é capaz de identificar e ajudar a validar vulnerabilidades de memória no PHP2Shell, mas com resultados que ainda incluem falsos positivos e falsos negativos e variam entre execuções, o que torna necessária a verificação experimental de suas conclusões.

O **objetivo geral** é avaliar as capacidades e limitações do OpenCode, utilizado como *harness* com modelos *open-weight*, na análise de vulnerabilidades de memória do PHP2Shell. Os **objetivos específicos** são:

- definir um ambiente controlado, com a configuração do OpenCode e dos modelos *open-weight* do catálogo da NVIDIA, e um conjunto de instruções padronizadas que permitam repetir os experimentos;
- medir a taxa de sucesso do agente na identificação e na validação das vulnerabilidades, quantificando falsos positivos e falsos negativos;
- medir o tempo de análise e o custo computacional, incluindo o consumo de *tokens* e o efeito dos limites de requisição do acesso gratuito;

- verificar a reprodutibilidade das respostas do agente em repetições sob as mesmas condições; e
- investigar, quando viável, aspectos arquiteturais da execução no simulador gem5, distinguindo-os da validação da vulnerabilidade.

O restante do artigo está organizado como segue. A Seção 2 discute os trabalhos relacionados, a Seção 3 descreve a metodologia e o ambiente experimental e a Seção 4 apresenta os trabalhos futuros.

2. Trabalhos Relacionados

3. Metodologia e Ambiente Experimental

A pesquisa adota uma abordagem experimental para avaliar a capacidade do agente de IA OpenCode na análise e validação de vulnerabilidades de memória no PHP2Shell, tendo a arquitetura RISC-V como ambiente-alvo. O experimento será dividido em etapas de preparação, análise assistida, execução controlada e avaliação dos resultados, conforme apresentado na Figura 1.

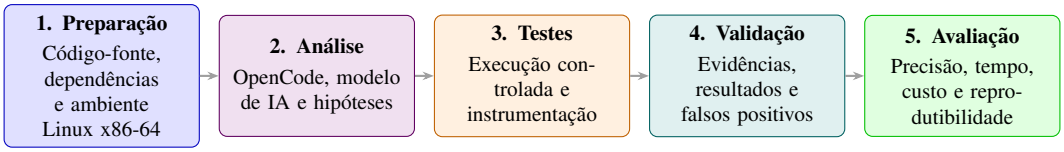


Figure 1. Fluxo do procedimento experimental.

Nas próximas subseções, são detalhados os recursos utilizados, a configuração do ambiente e os procedimentos adotados para a realização dos experimentos, desde a preparação do sistema até a análise dos resultados obtidos.

3.1. Ambiente e Procedimento Experimental

O experimento utilizará o OpenCode, integrado a um modelo de linguagem, para analisar o código-fonte do PHP2Shell e identificar possíveis vulnerabilidades de memória. Serão registradas as versões das ferramentas, as configurações do ambiente e as instruções fornecidas ao agente, de modo a manter condições consistentes entre as execuções.

A validação inicial será realizada em ambiente Linux com arquitetura x86-64, utilizada como referência por ser a plataforma documentada para a reprodução da cadeia de exploração do PHP2Shell. A arquitetura ARM64 será considerada para avaliar a portabilidade dos resultados, desde que haja suporte compatível às versões do software e aos componentes necessários. O uso de RISC-V poderá ser investigado posteriormente, mas não será tratado como requisito para a reprodução da vulnerabilidade.

O simulador gem5 será utilizado, quando viável, para executar programas de teste e investigar aspectos arquiteturais, como ciclos de processamento, instruções executadas e comportamento da hierarquia de memória. A execução integral do PHP2Shell no simulador dependerá da compatibilidade do sistema operacional, das bibliotecas e dos serviços necessários. Por isso, a validação da vulnerabilidade será distinguida dos experimentos de simulação arquitetural.

Os resultados produzidos pelo OpenCode serão comparados com as evidências obtidas nos testes controlados, permitindo avaliar a correção das análises e identificar falsos positivos. Todos os experimentos serão realizados em ambiente isolado, com registro das configurações, dos procedimentos e das limitações encontradas.

3.2. Métricas e Análise dos Resultados

A avaliação considerará a taxa de sucesso na identificação e validação das vulnerabilidades, o tempo de análise, o custo computacional e a reprodutibilidade. A taxa de sucesso será calculada pela proporção de tarefas validadas em relação ao total de tarefas válidas, conforme a Equação 1.

$$S = \frac{N_s}{N_v} \times 100 \tag{1}$$

em que N_s representa o número de tarefas bem-sucedidas e N_v o total de tarefas válidas.

O tempo será medido durante as execuções, enquanto o custo computacional considerará o consumo de memória, o tempo de CPU e, quando aplicável, o custo de utilização do modelo. A reprodutibilidade será verificada por meio de repetições sob as mesmas condições, comparando as respostas do OpenCode com as evidências obtidas nos testes. Os resultados serão organizados em tabelas e gráficos, destacando acertos, erros e limitações observadas.

4. Trabalhos Futuros

Como extensão, pretende-se substituir os modelos *open-weight* hospedados por modelos *open-source*, no sentido da definição da OSI [Open Source Initiative 2024], executados localmente. A motivação é dupla. A primeira é a privacidade: nessa configuração, o código analisado, as instruções e as respostas do agente não deixariam a máquina do pesquisador, o que permitiria análises totalmente anônimas e independentes de serviços de terceiros, de contas e de limites de requisição. A segunda é a transparência e a reprodutibilidade: com dados, código e parâmetros inspecionáveis e com a versão do modelo sob controle total, as execuções deixam de depender da disponibilidade e da política de um provedor.

Como o OpenCode é independente de modelo e oferece suporte a modelos locais [OpenCode 2026], essa extensão poderá manter o mesmo *harness* e as mesmas instruções padronizadas, alterando apenas o modelo. Será necessário investigar a viabilidade dessa abordagem, em especial os requisitos de hardware, como a memória de GPU, e o desempenho dos modelos locais em comparação com os hospedados, em termos de taxa de sucesso, tempo de análise e reprodutibilidade. Outras extensões incluem a avaliação em outras arquiteturas, como ARM64 e RISC-V, e uma exploração mais ampla do simulador gem5, conforme a compatibilidade do software.

References

[Addis et al. 2026] Addis, R., Porter, K., Ryan, J., Stull, M., Xu, S., Wagner, P., Honomichl, R., and Forsyth, S. (2026). Llm-based intelligent agents for cybersecurity: A tutorial and survey of automated vulnerability discovery. *IEEE Access*, 14:100884–100917.

[Gioacchini et al. 2025] Gioacchini, L., Delsanto, A., Drago, I., Mellia, M., Siracusano, G., and Bifulco, R. (2025). AutoPenBench: A vulnerability testing benchmark for generative agents. In Potdar, S., Rojas-Barahona, L., and Montella, S., editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1615–1624, Suzhou (China). Association for Computational Linguistics.

[He et al. 2026] He, L., Wu, S., Hao, H., Zhao, H., Yan, J., and Su, P. (2026). Reprobench: Benchmarking llm agents on reproducing vulnerability from scratch.

[Lee et al. 2025] Lee, H., Zhang, Z., Lu, H., and ZHANG, L. (2025). Sec-bench: Automated benchmarking of llm agents on real-world software security tasks. In Belgrave, D., Zhang, C., Lin, H., Pascanu, R., Koniusz, P., Ghassemi, M., and Chen, N., editors, *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 116342–116378. Curran Associates, Inc.

[NVIDIA 2026a] NVIDIA (2026a). deepseek-v4.1-flash (deepseek-ai). build.nvidia.com. Acesso em: 11 nov. 2026.

[NVIDIA 2026b] NVIDIA (2026b). glm-5-3-flash (z-ai). build.nvidia.com. Acesso em: 11 nov. 2026.

[NVIDIA 2026c] NVIDIA (2026c). kimi-k3.1-pro (moonshotai). build.nvidia.com. Acesso em: 11 nov. 2026.

[NVIDIA 2026d] NVIDIA (2026d). NVIDIA NIM FAQ. NVIDIA Developer Forums. Acesso em: 11 nov. 2026.

[NVIDIA 2026e] NVIDIA (2026e). NVIDIA NIM model catalog. build.nvidia.com. Acesso em: 11 nov. 2026.

[Open Source Initiative 2024] Open Source Initiative (2024). The open source AI definition – 1.0. Open Source Initiative. Acesso em: 11 nov. 2026.

[OpenAI 2026] OpenAI (2026). Patch the planet: global response coordination as agents uncover critical vulnerabilities. OpenAI. Acesso em: 11 nov. 2026.

[OpenCode 2026] OpenCode (2026). OpenCode: the open source coding agent. Site oficial. Acesso em: 11 nov. 2026.

[PHP Group 2026a] PHP Group (2026a). RCE via OOM corruption (CVE-2026-109618). GitHub Security Advisory GHSA-w8hw-w9fx-j79w, php/php-src. Acesso em: 11 nov. 2026.

[PHP Group 2026b] PHP Group (2026b). RCE via stale global state (CVE-2026-109619). GitHub Security Advisory GHSA-6wmf-4rv5-p3rp, php/php-src. Acesso em: 11 nov. 2026.

[Shi et al. 2026] Shi, T., Rheem, R., Jiang, D., Wang, M., De La Riega, F., Wang, Z., Jiang, J., Cheung, A., Tai, S., Cha, J., Tu, J., Han, G., Wang, C., He, J., Guo, W., and Song, D. (2026). CyberGym-E2E: Scalable real-world benchmark for AI agents' end-to-end cybersecurity capabilities. In Zhang, T., Dudik, M., Jaggi, M., Agarwal, A., Li, S., Schuurmans, D., Zhu, J., Berkenkamp, F., Dong, H., and Bietti, A., editors, *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 112094–112108. PMLR.

[Tzadik et al. 2026] Tzadik, S., Tamari, S., Ohfeld, N., and Schwartz, T. (2026). PHP2Shell: AI uncovers critical RCE vulnerabilities affecting 70% of the internet. Wiz Blog. Acesso em: 11 nov. 2026.

[Zhu et al. 2025] Zhu, Y., Kellermann, A., Bowman, D., Li, P., Gupta, A., Danda, A., Fang, R., Jensen, C., Ihli, E., Benn, J., Geronimo, J., Dhir, A., Rao, S., Yu, K., Stone, T., and Kang, D. (2025). CVE-bench: A benchmark for AI agents' ability to exploit real-world web application vulnerabilities. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J., editors, *Proceedings of the*

42nd International Conference on Machine Learning, volume 267 of *Proceedings of Machine Learning Research*, pages 79850–79867. PMLR.